SENTINELNET – AI-POWERED NETWORK INTRUSION DETECTION SYSTEM (NIDS)

Author: Upasana Prabhakar

Mentor: Dr. N Jagan Mohan

ABSTRACT: SentinelNet aims to build a smart Network Intrusion Detection System (NIDS) powered by AI and machine learning. The central goal is to automatically identify and respond to suspicious or potentially harmful network traffic and cyber-attacks in real time. Using historical data, machine learning models will be trained to distinguish between normal and malicious activities, extract crucial features, classify threats, and trigger alerts to help network defenders respond quickly and effectively.

KEYWORDS: Network Intrusion Detection System (NIDS), Cybersecurity, Machine Learning, Artificial Intelligence, Binary Classification, Multi-Class Classification, NSL-KDD Dataset, CICIDS2017 Dataset, Ensemble Methods, Random Forest, Gradient Boosting, Decision Tree, Logistic Regression, Feature Engineering, Class Imbalance, Anomaly Detection, Signature-Based Detection, Cyber Threat Intelligence, Network Security, Real-Time Detection, Attack Classification, DoS Attack, Probe Attack, U2R Attack, R2L Attack, Performance Metrics, ROC Curve, Confusion Matrix, SentinelNet

STATEMENT OF ORIGINALITY: This paper presents SentinelNet, a hybrid IDS combining signature-based and anomaly-based methods with advanced preprocessing, feature selection, and class-imbalance handling, applied to NSL-KDD and CICIDS2017 datasets.

1 INTRODUCTION

SentinelNet aims to build a smart Network Intrusion Detection System (NIDS) powered by AI and machine learning. The central goal is to automatically identify and respond to suspicious or potentially harmful network traffic and cyber-attacks in real time.

Using historical data, machine learning models will be trained to:

- Distinguish between normal and malicious activities
- Extract crucial features
- Classify threats
- Trigger alerts to help network defenders respond quickly and effectively

2 LITERATURE

The field of Intrusion Detection Systems (IDS) has evolved from rule-based methods to AI-powered frameworks. Literature highlights:

- **Signature-Based IDS:** Relies on predefined attack signatures, effective for known threats but fails on zero-day attacks.
- **Anomaly-Based IDS:** Uses statistical models and ML to detect unusual traffic patterns, can detect unknown threats but may have higher false positives.

2.1 Benchmark datasets shaping IDS research

- KDD Cup 1999: Early dataset, criticized for redundancy and outdated attacks.
- NSL-KDD: Improved version, duplicates removed, balanced dataset for academic evaluation.
- CICIDS2017: Realistic enterprise traffic, modern attacks like DDoS, infiltration, and web exploits.

2.2 ML algorithms widely used

Decision Trees, Random Forests, SVMs, Neural Networks, Deep Learning, Ensemble methods, Hybrid IDS models.

2.3 Insights

- Feature selection and preprocessing are critical.
- Class imbalance is a challenge.
- Hybrid approaches combine speed of signature-based and intelligence of anomaly-based IDS.

3 METHODOLOGY

3.1 Database

3.1.1 NSL-KDD Dataset

A refined dataset to reduce redundancy:

- 41 features: basic (protocol, service, bytes) + traffic-level (flags, errors)
- Training: 125,973 records
- Test: 22,544 records
- Attack types: DoS (smurf, neptune), Probe (portsweep, nmap), U2R (buffer overflow, rootkit), R2L (ftp write, guess passwd)

3.1.2 CICIDS2017 Dataset

Realistic enterprise network traffic:

- 78–83 features: flow, statistical, packet-level
- Over 2.8 million labeled flow records across 5 days
- Attack types: DDoS (Hulk, GoldenEye, Slowloris), Brute Force, Web Attacks (SQLi, XSS), Botnet/Infiltration, Reconnaissance (PortScan, Heartbleed)

3.1.3 Dataset Comparison

Table 1: Dataset Comparison

Attribute	NSL-KDD	CICIDS2017
No. of Features	41	78–83
Total Records	$\sim$150,000	$>$2.8 million
Traffic	Simulated, old-generation	Realistic, enterprise-scale
Usage	Lightweight model evaluation	Practical deployment
Attack Variety	4 legacy categories	14+ modern attacks

3.2 Preprocessing

The preprocessing pipeline applied in SentinelNet ensures clean, balanced, and representative data for ML models. Steps include:

- Handling missing values using imputation or removal of corrupted records.
- Feature scaling with MinMaxScaler and StandardScaler depending on algorithm needs.
- Encoding categorical features with One-Hot Encoding for protocol types, services, and flags.
- Correlation analysis to detect redundant features; highly correlated attributes were dropped to prevent multicollinearity.
- Addressing class imbalance with:
  - **Power sampling** to balance majority/minority classes.
  - **Undersampling** majority classes where applicable, to prevent bias towards normal traffic.
- Stratified train-test split to preserve class distributions.

3.2.1 Preprocessing Techniques Not Used

Some common preprocessing methods were considered but not applied in this work:

- **Min–Max Normalization:** Not used since standardization (z-score scaling) provided better handling of outliers and ensured feature comparability.
- **SMOTE (Synthetic Minority Oversampling Technique):** Avoided to prevent introducing synthetic traffic samples that might not reflect realistic attack behavior.
- **Label Encoding:** Not applied because categorical variables had no ordinal relationship; one-hot encoding was more appropriate.
- **Log Transformation:** Skipped since most continuous features did not exhibit heavy skewness requiring transformation.

By excluding these methods, the preprocessing pipeline remained tailored to the dataset characteristics while avoiding unnecessary complexity.

3.2.2 Scaling vs Normalization

Table 2: Scaling vs Normalization

Aspect	Standardization	Normalization
Definition	Transforms features to mean=0, std=1	Rescales to [0,1]
Purpose	For Gaussian-based algorithms	For bounded-input algorithms
Use Cases	SVM, Logistic Regression, PCA	Neural Nets, KNN
Effect on Outliers	Less sensitive	Sensitive (compressed to 0–1)
Preserves Data Shape	Yes	Yes, but centers differently
When to Use	Gaussian-based models	NN, image data

3.3 Proposed Method: Logistic Regression

Logistic Regression is a model that predicts the probability of a data point belonging to a class using a simple formula:

$$P = \frac{1}{1 + e^{-z}}, \quad \text{where } z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Here, x_i are input features, w_i are weights, and b is the bias. The output probability is then converted to a class label (commonly using 0.5 as the threshold). Logistic Regression is simple, interpretable, and works well for linearly separable data, but may underperform for complex non-linear patterns in IDS.

4 RESULTS

4.1 NSL-KDD Binary Classification Results

4.1.1 Training vs Testing Accuracy (Binary)

Table 3: NSL-KDD Binary Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	97.85	81.88
Decision Tree	99.84	86.48
Random Forest	99.93	86.66
Gradient Boosting	99.98	86.20

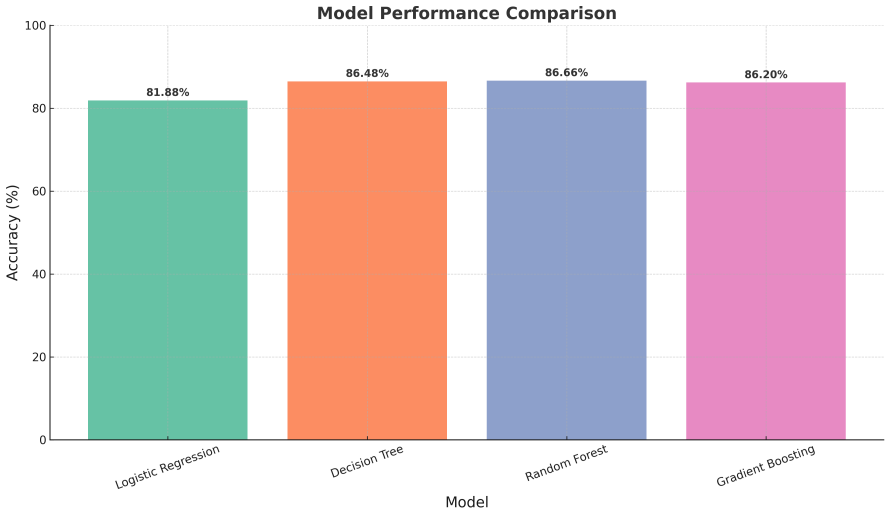

Figure 1: NSL-KDD Binary Classification: Model accuracy comparison showing that Random Forest slightly outperforms other models, while Logistic Regression performs lowest, highlighting the difference in handling nonlinear attack traffic.

4.1.2 Macro Average Performance (Binary)

Table 4: NSL-KDD Binary Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.83	0.82	0.82
Decision Tree	0.88	0.86	0.86
Random Forest	0.89	0.86	0.86
Gradient Boosting	0.88	0.86	0.86

4.1.3 Macro Average Specificity and Time Taken (Binary)

Table 5: NSL-KDD Binary Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9840	1.57
Decision Tree	0.9977	0.72
Random Forest	0.9997	2.73
Gradient Boosting	0.9991	43.09

4.1.4 Confusion Matrix Explanation

The confusion matrix provides detailed insights into classification:

Table 6: Structure of a Confusion Matrix

	Predicted Normal	**Predicted Attack**
Actual Normal	True Negative (TN)	False Positive (FP)
Actual Attack	False Negative (FN)	True Positive (TP)

Interpretation:

- TN: Correctly identified benign traffic.
- FP: Benign traffic misclassified as attack (false alarm).

- FN: Attack traffic missed (dangerous miss).
- TP: Correctly detected attack.

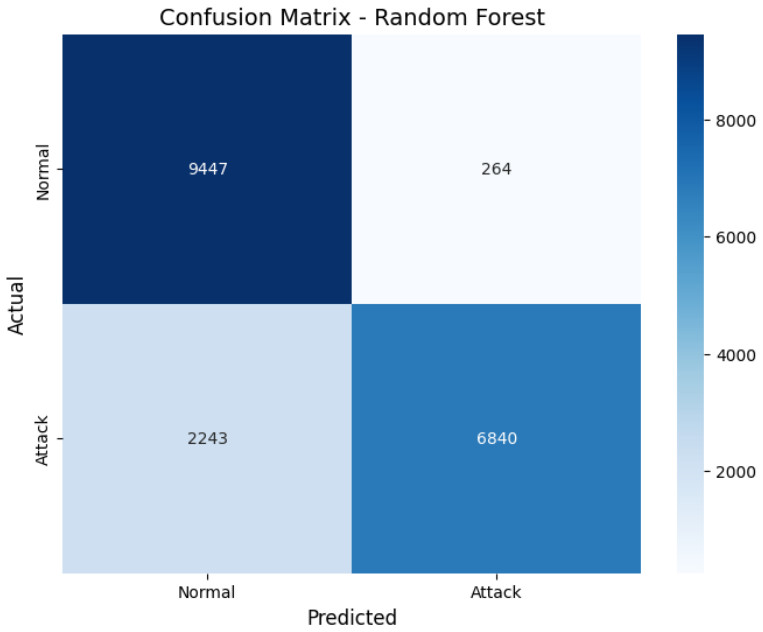

Figure 2: NSL-KDD Binary Classification: Confusion matrix visualization showing classification distribution, with emphasis on how well the models minimize false positives and false negatives.

4.1.5 ROC Curve (Binary)

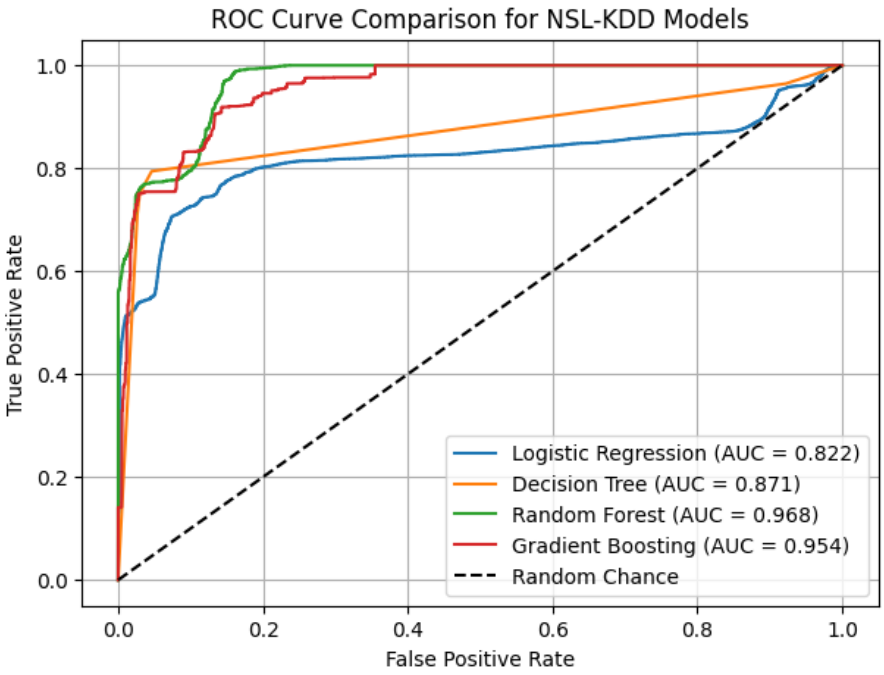

Figure 3: NSL-KDD Binary Classification: ROC curves showing the trade-off between sensitivity and specificity for each model, with Random Forest achieving the best balance.

4.1.6 Model Comparison Analysis (Binary)

The NSL-KDD binary classification shows that Random Forest achieves the best balance between accuracy and F1-score, while Gradient Boosting slightly underperforms compared to expectations. Logistic Regression remains interpretable but

limited for complex non-linear attack detection.

—

4.2 NSL-KDD Multi-Class Classification Results

4.2.1 Training vs Testing Accuracy (Multi-Class)

Table 7: NSL-KDD Multi-Class Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	98.79	82.54
Decision Tree	99.99	86.27
Random Forest	99.99	86.59
Gradient Boosting	99.91	86.54

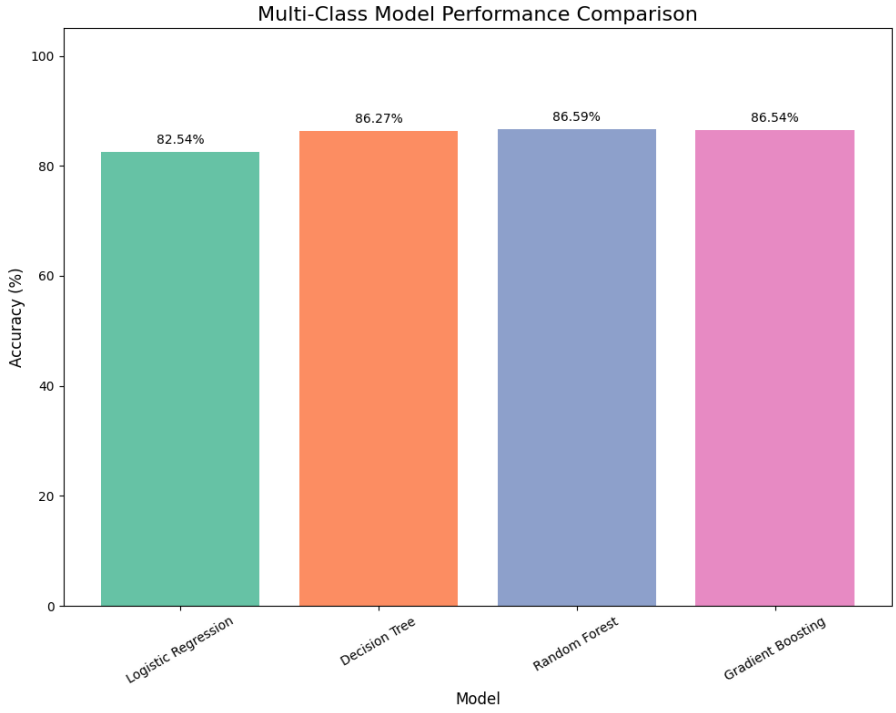

Figure 4: NSL-KDD Multi-Class Classification: Model accuracy comparison showing ensemble methods achieving superior performance over Logistic Regression in multi-class attack type classification.

4.2.2 Macro Average Performance (Multi-Class)

Table 8: NSL-KDD Multi-Class Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.60	0.56	0.53
Decision Tree	0.66	0.64	0.62
Random Forest	0.61	0.64	0.61
Gradient Boosting	0.66	0.65	0.63

4.2.3 Macro Average Specificity and Time Taken (Multi-Class)

Table 9: NSL-KDD Multi-Class Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9984	4.78
Decision Tree	0.9997	1.55
Random Forest	0.9998	8.04
Gradient Boosting	0.9997	270.39

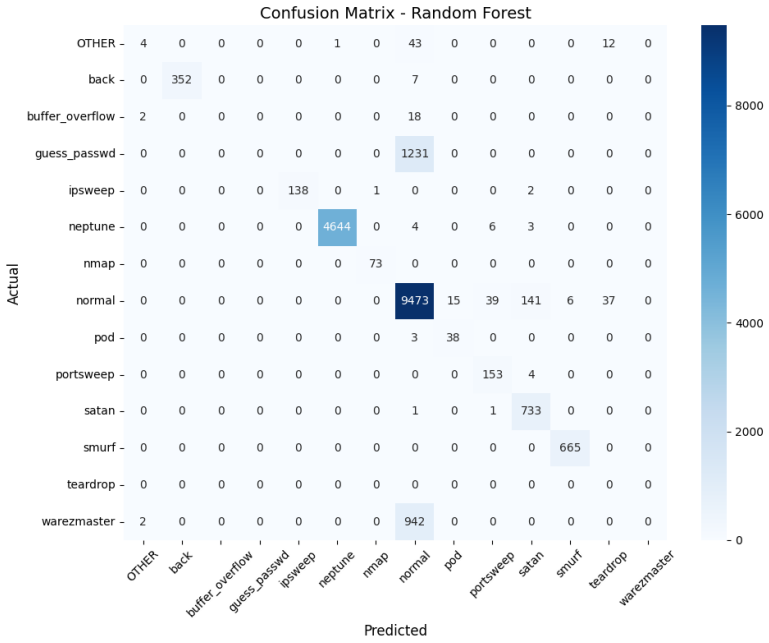

Figure 5: NSL-KDD Multi-Class Classification: Confusion matrix visualization showing the distribution of predictions across different attack types and normal traffic.

4.2.4 ROC Curve (Multi-Class)

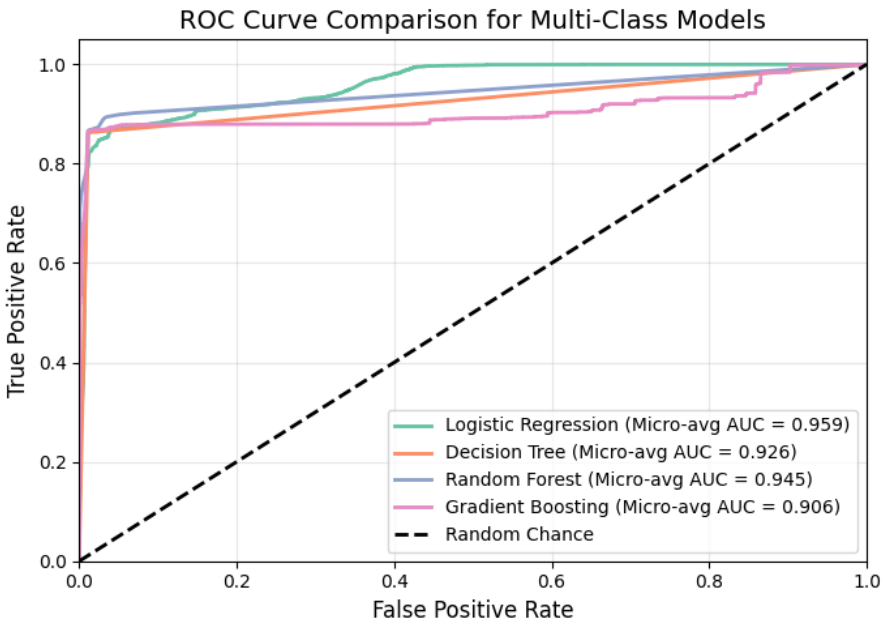

Figure 6: NSL-KDD Multi-Class Classification: ROC curves displaying the classification performance across multiple attack categories.

4.2.5 Model Comparison Analysis (Multi-Class)

The NSL-KDD multi-class classification demonstrates that while ensemble methods maintain high accuracy, the precision and recall values are lower compared to binary classification due to the increased complexity of distinguishing between multiple attack types. Gradient Boosting shows the best overall performance in terms of balanced metrics.

—

4.3 CICIDS2017 Binary Classification Results

4.3.1 Training vs Testing Accuracy (Binary)

Table 10: CICIDS2017 Binary Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	96.92	97.04
Decision Tree	99.06	99.08
Random Forest	99.78	99.77
Gradient Boosting	99.83	99.83

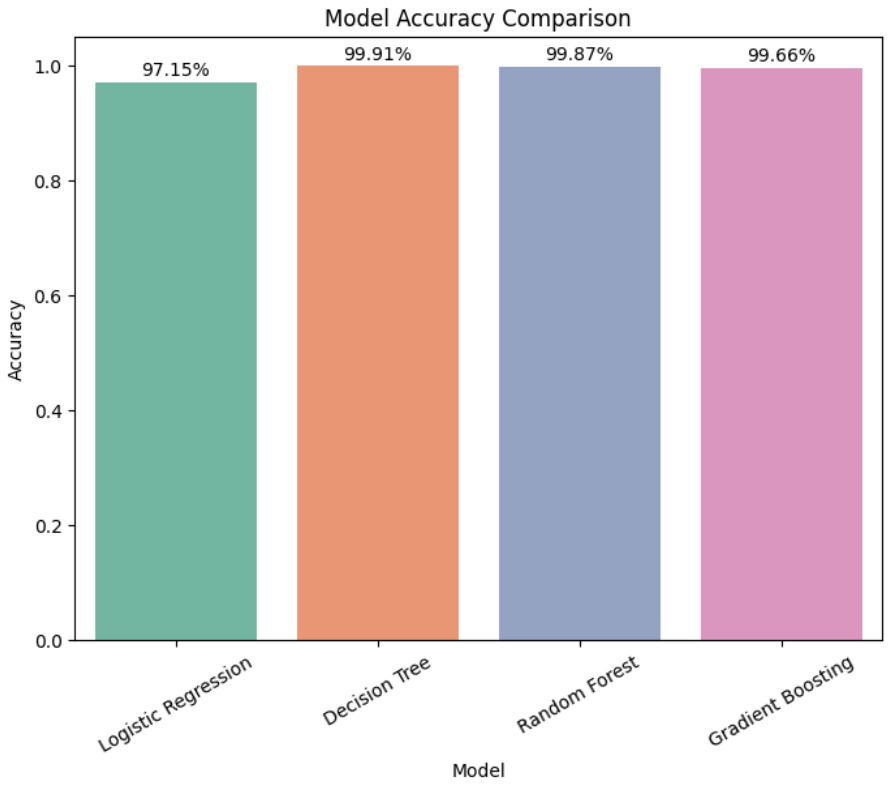

Figure 7: CICIDS2017 Binary Classification: Model accuracy comparison showing all ensemble methods achieve near-perfect accuracy, clearly outperforming Logistic Regression.

4.3.2 Macro Average Performance (Binary)

Table 11: CICIDS2017 Binary Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.96	0.97	0.97
Decision Tree	0.99	0.99	0.99
Random Forest	1.00	1.00	1.00
Gradient Boosting	1.00	1.00	1.00

4.3.3 Macro Average Specificity and Time Taken (Binary)

Table 12: CICIDS2017 Binary Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9933	5.53
Decision Tree	0.9993	15.04
Random Forest	0.9990	97.27
Gradient Boosting	0.9991	346.73

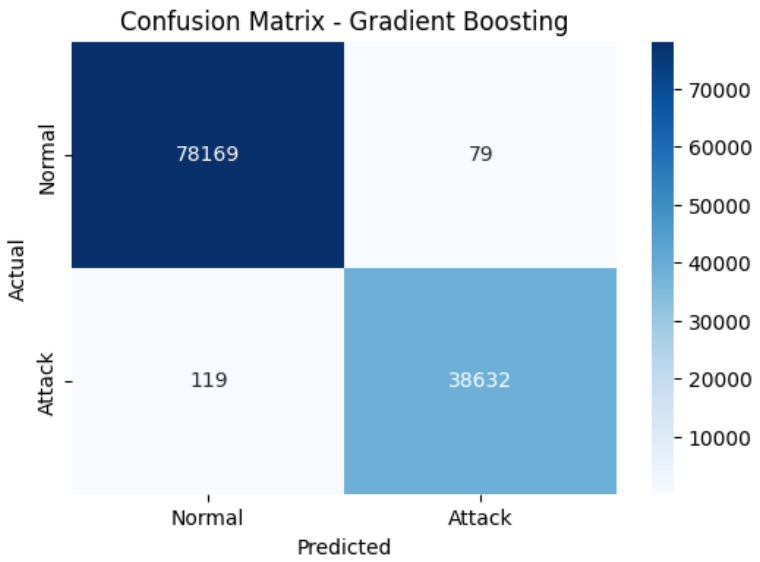

Figure 8: CICIDS2017 Binary Classification: Confusion matrix visualization showing that ensemble models nearly eliminate false negatives and false positives, indicating high robustness.

4.3.4 ROC Curve (Binary)

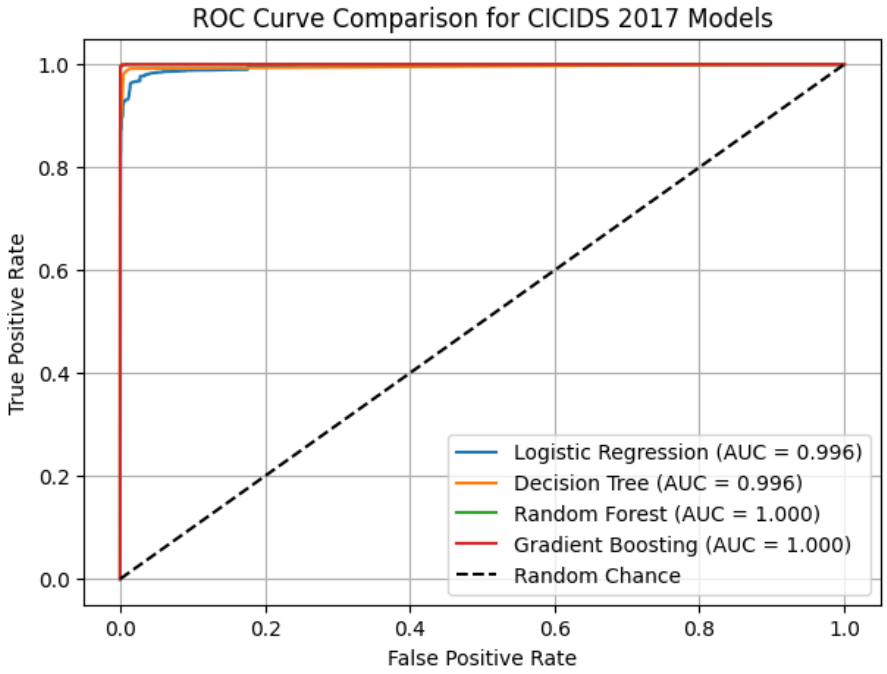

Figure 9: CICIDS2017 Binary Classification: ROC curves displaying almost perfect classification capability for Random Forest and Gradient Boosting, closely followed by Decision Tree and Logistic Regression.

4.3.5 Model Comparison Analysis (Binary)

The CICIDS2017 binary classification confirms that ensemble methods (Random Forest, Gradient Boosting) perform nearly perfectly, reflecting their robustness on modern attack traffic. Logistic Regression, while performing well, lags slightly due to its linear assumptions.

—

4.4 CICIDS2017 Multi-Class Classification Results

4.4.1 Training vs Testing Accuracy (Multi-Class)

Table 13: CICIDS2017 Multi-Class Classification: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	94.11	94.21
Decision Tree	99.40	99.41
Random Forest	98.83	98.82
Gradient Boosting	99.90	99.87

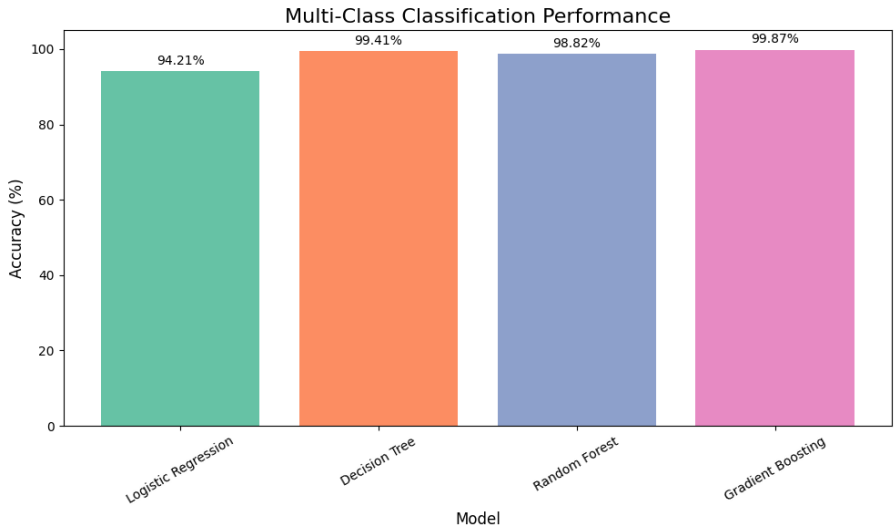

Figure 10: CICIDS2017 Multi-Class Classification: Model accuracy comparison showing Gradient Boosting achieving the highest accuracy, followed by Decision Tree, with all ensemble methods significantly outperforming Logistic Regression.

4.4.2 Macro Average Performance (Multi-Class)

Table 14: CICIDS2017 Multi-Class Classification: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.66	0.97	0.76
Decision Tree	0.99	0.89	0.93
Random Forest	0.92	0.94	0.91
Gradient Boosting	0.99	0.99	0.99

4.4.3 Macro Average Specificity and Time Taken (Multi-Class)

Table 15: CICIDS2017 Multi-Class Classification: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9933	16.55
Decision Tree	0.9993	6.51
Random Forest	0.9990	12.28
Gradient Boosting	0.9991	1133.07

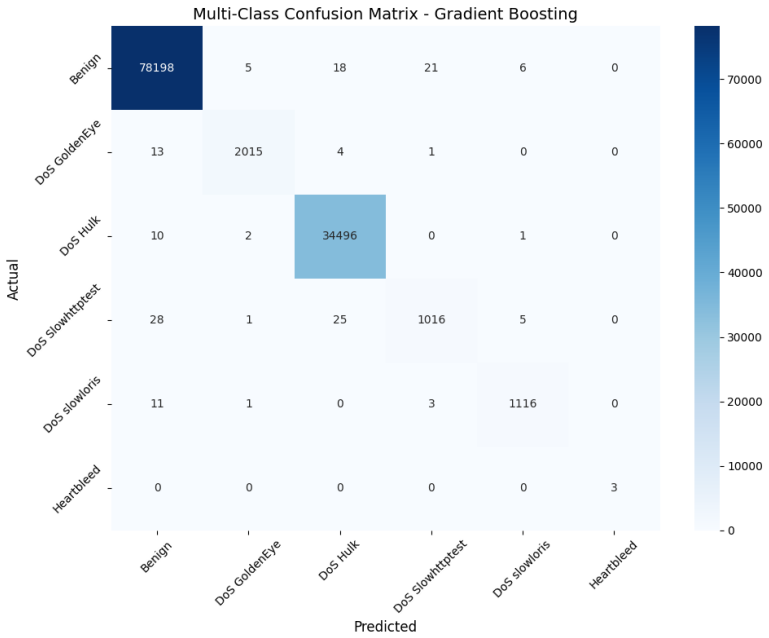

Figure 11: CICIDS2017 Multi-Class Classification: Confusion matrix visualization showing the distribution of predictions across different DoS attack variants and benign traffic, with Gradient Boosting achieving near-perfect classification.

4.4.4 ROC Curve (Multi-Class)

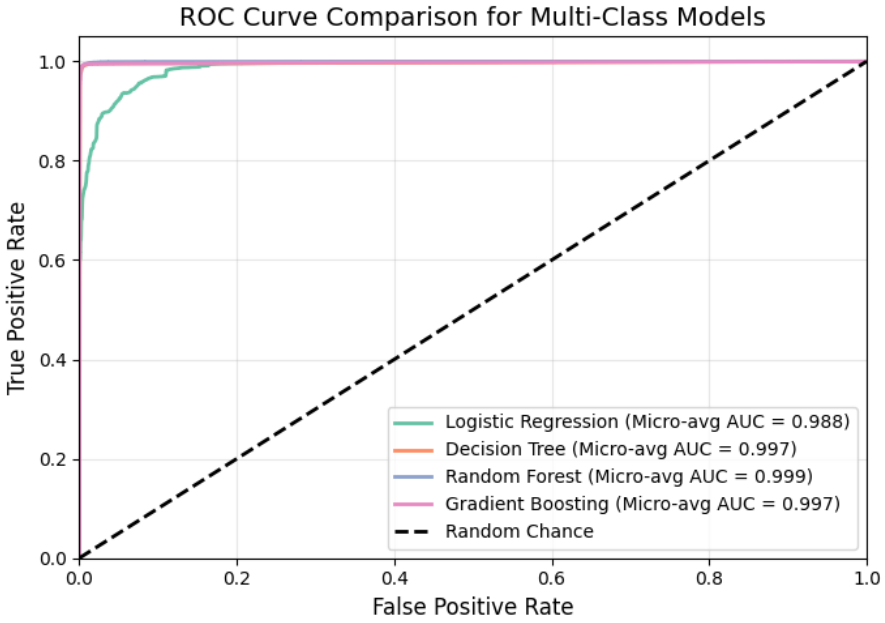

Figure 12: CICIDS2017 Multi-Class Classification: ROC curves displaying excellent classification performance across multiple attack categories, with Gradient Boosting leading.

4.4.5 Model Comparison Analysis (Multi-Class)

The CICIDS2017 multi-class classification reveals that Gradient Boosting achieves outstanding performance with near-perfect metrics across all attack types. Decision Tree shows strong balanced performance, while Random Forest maintains high accuracy with slightly lower precision for minority classes. Logistic Regression struggles with multi-class complexity, achieving high recall but lower precision.

—

4.5 NSL-KDD Binary Classification Final Results

Table 16: NSL-KDD Binary Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9700	0.9840	0.83	0.82	0.902	97.85%	97.76%	81.88%	1.57
Decision Tree	0.9977	0.9977	0.88	0.86	0.807	99.84%	99.77%	86.48%	0.72
Random Forest	0.9971	0.9997	0.89	0.86	0.931	99.93%	99.86%	86.66%	2.73
Gradient Boosting	0.9991	0.9991	0.88	0.86	0.930	99.98%	99.90%	86.20%	43.09

4.6 NSL-KDD Multi-Class Classification Final Results

Table 17: NSL-KDD Multi-Class Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.8309	0.9984	0.60	0.53	0.915	98.79%	98.82%	82.54%	4.78
Decision Tree	0.9348	0.9997	0.66	0.62	0.967	99.99%	99.78%	86.27%	1.55
Random Forest	0.9157	0.9998	0.61	0.61	0.957	99.99%	99.83%	86.59%	8.04
Gradient Boosting	0.8925	0.9997	0.66	0.63	0.946	99.91%	99.81%	86.54%	270.39

4.7 CICIDS2017 Binary Classification Final Results

Table 18: CICIDS2017 Binary Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9700	0.9933	0.96	0.97	0.994	96.92%	97.21%	97.04%	5.53
Decision Tree	0.9900	0.9993	0.99	0.99	0.996	99.06%	99.92%	99.08%	15.04
Random Forest	1.0000	0.9990	1.00	1.00	0.999	99.78%	99.86%	99.77%	97.27
Gradient Boosting	1.0000	0.9991	1.00	1.00	0.999	99.83%	99.70%	99.83%	346.73

4.8 CICIDS2017 Multi-Class Classification Final Results

Table 19: CICIDS2017 Multi-Class Classification: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.9694	0.9933	0.66	0.76	0.981	94.11%	94.01%	94.21%	16.55
Decision Tree	0.8859	0.9993	0.99	0.93	0.943	99.40%	99.39%	99.41%	6.51
Random Forest	0.9355	0.9990	0.92	0.91	0.967	98.83%	98.82%	98.82%	12.28
Gradient Boosting	0.9870	0.9991	0.99	0.99	0.993	99.90%	99.87%	99.87%	1133.07

5 CONCLUSION

The evaluation of SentinelNet on NSL-KDD and CICIDS2017 datasets highlights the importance of ensemble learning methods in intrusion detection, with distinct insights from binary and multi-class classification approaches.

Binary Classification Insights:

- On NSL-KDD binary classification, Random Forest achieved the best overall performance with high accuracy and balanced precision-recall.
- Simple attack vs. normal traffic classification showed strong performance across all ensemble methods.
- CICIDS2017 binary classification demonstrated near-perfect results for Random Forest and Gradient Boosting, confirming their effectiveness on modern attack patterns.

Multi-Class Classification Insights:

- NSL-KDD multi-class classification revealed the increased complexity of distinguishing between specific attack types.

- While accuracy remained high, precision and recall values decreased due to class imbalance and attack type similarities.
- CICIDS2017 multi-class classification showed exceptional performance by Gradient Boosting (99.87% accuracy), effectively distinguishing between multiple DoS attack variants.
- Decision Tree demonstrated strong balanced performance with high precision (0.99) in multi-class scenarios.

Overall Findings:

- Gradient Boosting emerged as the most accurate model for multi-class classification on both datasets, though at the cost of significantly higher training time (270.39s for NSL-KDD, 1133.07s for CICIDS2017).
- Random Forest provides an optimal trade-off between accuracy, training time, and interpretability for practical NIDS deployment.
- Logistic Regression, while computationally efficient, underperformed compared to non-linear ensemble models on both datasets and classification types, particularly in multi-class scenarios.
- Multi-class classification provides more granular threat intelligence but requires careful handling of class imbalance and computational resources.
- CICIDS2017's realistic enterprise traffic patterns enabled better model performance compared to NSL-KDD's simulated data.

This study confirms that ensemble models, particularly Random Forest and Gradient Boosting, are essential for modern NIDS deployment. The choice between binary and multi-class classification depends on operational requirements: binary classification offers faster detection with high accuracy, while multi-class classification provides detailed threat categorization at the cost of additional computational overhead.

6 ACKNOWLEDGEMENTS

We would like to thank our mentor, project guides, and dataset providers for their support throughout this research. Special thanks go to the Infosys Springboard initiative for providing the opportunity to develop and present this project. We also acknowledge the open-source research community for making datasets like NSL-KDD and CICIDS2017 publicly available, which made experimentation possible.